

ShipFix Frontend → Backend Integration Roadmap

Project: SwarmCode (Frontend) → ShipFix (Backend)

Date: December 6, 2025

Architecture Overview

Your Current Stack

Frontend:

- Pure HTML/CSS/JavaScript (Static files)
- Tailwind CSS (CDN)
- Font Awesome icons
- Custom animations & design system

Backend (Already Built):

- Next.js 14 App Router
 - TypeScript
 - Prisma + PostgreSQL
 - AI Agent system (Scanner, Tester, Fixer, Refactor)
 - Constitution builder
 - Tier system (Free, Pro, Enterprise)
-

Integration Strategy

Option 1: Replace All UI Pages (RECOMMENDED)

Replace ShipFix's existing UI pages with your custom design while keeping all backend APIs intact.

What to Keep:

-  All `/api/*` routes
-  All `lib/*` utilities
-  All agent logic
-  Database schema
-  Authentication system

What to Replace:

-  `/app/page.tsx` (landing)
-  `/app/dashboard/*` pages
-  `/app/login/*` pages
-  Existing components

What to Add:

-  Your custom pages (converted to React/Next.js)

- ✨ Your design system as Tailwind config
 - ✨ Your custom animations
-

Page-by-Page Integration Map

1. Landing Page (swarm-index.txt → app/page.tsx)

Current Features in Your Design:

- Hero section with orbit animation
- Four agent cards (Scanner, Tester, Fixer, Refactor)
- Stats section (98%, 10x, 500k, 24/7)
- Process flow (4 steps)
- Features grid
- Pricing tiers (\$0, \$99, Custom)
- Terminal demo with live code

Backend APIs Needed:

✗ None (Pure marketing page)

Action Items:

1. Convert HTML → `app/page.tsx` (React component)
 2. Move CSS → `tailwind.config.ts`
 3. Add smooth scroll behavior
 4. Link “Launch App” button → `/dashboard`
 5. Link “Start Free Trial” → `/signup`
-

2. Dashboard (swarm-app.txt → app/dashboard/page.tsx)

Current Features in Your Design:

- 4 swarm status cards (Scanner, Tester, Fixer, Refactor)
- Live activity feed
- Issues list with severity badges
- Code health score (75/100)
- Quick stats grid
- Recent files sidebar
- Terminal preview

Backend APIs Needed:

✓ Already Exists:

```
// Get all bugs/issues
GET /api/bugs?organizationId={id}&status={status}

// Get agent activity logs
GET /api/agents/activity?bugId={id}

// Trigger agents
POST /api/agents/triage (Scanner equivalent)
POST /api/agents/diagnostic (Scanner ☐ Fixer)
POST /api/agents/review (Review before merge)
```

✗ Need to Build:

```
// Get real-time swarm status
GET /api/swarms/status
// Response: { scanner: { status: 'running', progress: 67 }, ... }

// Get code health metrics
GET /api/repositories/{id}/health
// Response: { overallScore: 75, security: 60, reliability: 85, ... }

// Get live activity feed
GET /api/activity/feed?limit=10
// Response: [{ agent: 'scanner', message: '...', timestamp: '...' }]
```

Action Items:

1. Convert HTML → `app/dashboard/page.tsx`
2. Create `useSwarmStatus()` hook for real-time updates
3. Create `<ActivityFeed />` component
4. Wire up “Deploy Swarm” button → POST `/api/scan/full`
5. Add Server-Sent Events (SSE) for real-time progress

3. Repositories Page (`swarm-repository.txt` → `app/dashboard/repositories/page.tsx`)

Current Features in Your Design:

- Repository cards with health scores
- Language breakdown bars
- Stats (issues, coverage, PRs ready)
- Filter tabs (All, Active, Needs Attention, Archived)
- “Run Swarm” and “Settings” buttons per repo

Backend APIs Needed:

✓ Already Exists:

```
// None directly (we have Bugs table but no Repositories table yet)
```

✗ Need to Build:

```
// Get all repositories
GET /api/repositories
// Response: [{ id, name, owner, healthScore, issues, coverage, ... }]

// Get repository details
GET /api/repositories/{id}

// Connect new repository
POST /api/repositories/connect
// Body: { gitUrl, provider: 'github' | 'gitlab' }

// Run swarm on repository
POST /api/repositories/{id}/scan

// Get repository health score
GET /api/repositories/{id}/health
```

Database Changes Needed:

```
model Repository {
  id          String    @id @default(cuid())
  organizationId String
  name        String
  owner       String
  provider     String    // 'github' | 'gitlab' | 'bitbucket'
  gitUrl       String
  mainBranch   String    @default("main")
  healthScore  Int        @default(0)
  lastScanAt   DateTime?
  filesCount   Int        @default(0)
  issuesCount  Int        @default(0)
  coverage     Float      @default(0)
  languages    Json       // ["TypeScript": 55, "JavaScript": 30, ... ]
  status       String    @default("active") // active | archived

  bugs         Bug[]
  organization  Organization @relation(fields: [organizationId], references: [id])

  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt

  @@index([organizationId])
}
```

Action Items:

1. Add `Repository` model to Prisma schema
 2. Run `yarn prisma migrate dev`
 3. Create `/api/repositories/*` routes
 4. Convert HTML → React component
 5. Wire up “Add Repository” modal
 6. Connect “Run Swarm” → POST `/api/repositories/{id}/scan`
-

4. Swarms Page (swarm-swarm.txt → app/dashboard/swarms/page.tsx)

Current Features in Your Design:

- 4 large swarm cards with:
- Toggle switches (on/off)
- Live stats (Issues Found, Files Scanned, Avg Runtime)
- Progress bars
- Start/Stop buttons
- Swarm Configuration panel:
- Auto-deploy on push
- Create PRs automatically
- Security-first mode
- Slack/Email notifications
- Conservative mode

Backend APIs Needed:

✓ Already Exists:

```
// Trigger agents (partially)
POST /api/agents/triage
POST /api/agents/diagnostic
POST /api/agents/review
```

✗ Need to Build:

```
// Get swarm configuration
GET /api/swarms/config
// Response: { autoDeployOnPush: true, createPRsAuto: true, ... }

// Update swarm configuration
PATCH /api/swarms/config
// Body: { autoDeployOnPush: false, ... }

// Toggle swarm on/off
POST /api/swarms/{swarmType}/toggle
// Body: { enabled: true }

// Get swarm stats
GET /api/swarms/{swarmType}/stats
// Response: { issuesFound: 23, filesScanned: 847, avgRuntime: '5m' }

// Start/Stop swarm
POST /api/swarms/{swarmType}/start
POST /api/swarms/{swarmType}/stop
```

Database Changes Needed:

```
model SwarmConfig {
  id                String    @id @default(cuid())
  organizationId    String    @unique

  // Swarm toggles
  scannerEnabled    Boolean    @default(true)
  testerEnabled     Boolean    @default(true)
  fixerEnabled       Boolean    @default(false)
  refactorEnabled    Boolean    @default(false)

  // Automation settings
  autoDeployOnPush  Boolean    @default(false)
  createPRsAutomatic Boolean    @default(true)
  securityFirstMode Boolean    @default(true)
  conservativeMode  Boolean    @default(false)

  // Notifications
  slackNotifications Boolean    @default(false)
  slackWebhook        String?
  emailDigest          Boolean    @default(true)

  organization       Organization @relation(fields: [organizationId], references: [id])

  createdAt          DateTime @default(now())
  updatedAt           DateTime @updatedAt
}
```

Action Items:

1. Add `SwarmConfig` model to Prisma schema
2. Create `/api/swarms/*` routes
3. Convert HTML → React component
4. Add toggle switch logic
5. Wire up configuration saves
6. Add real-time stats updates

5. Pull Requests Page (swarm-pullrequests.txt → app/dashboard/pull-requests/page.tsx)

Current Features in Your Design:

- PR list sidebar with:
 - Checkboxes for batch merge
 - Status badges (Security, Tests, Refactor, Cleanup)
 - Agent attribution (which swarm created it)
 - +/- line counts
- PR detail view with:
 - File diff viewer
 - Code syntax highlighting
 - AI explanation from swarm
 - Merge/Close buttons

- Checks status (Tests passing, No conflicts, etc.)

Backend APIs Needed:

✅ Already Exists:

```
// Bug review (partial)
GET /api/bugs/{id}
// Has reviewResult with test scenarios & deployment checklist
```

❌ Need to Build:

```
// Get all PRs
GET /api/pull-requests
// Response: [{ id, title, description, agent, status, diff, ... }]

// Get PR details
GET /api/pull-requests/{id}
// Response: { id, title, diff: [{ file, additions, deletions }], ... }

// Merge PR
POST /api/pull-requests/{id}/merge

// Close PR
POST /api/pull-requests/{id}/close

// Batch merge PRs
POST /api/pull-requests/batch-merge
// Body: { prIds: ['pr1', 'pr2'] }
```

Database Changes Needed:

```

model PullRequest {
  id          String   @id @default(cuid())
  repositoryId String
  bugId       String?

  prNumber    Int
  title       String
  description  String   @db.Text
  branch      String
  targetBranch String   @default("main")

  agent      String // 'scanner' | 'tester' | 'fixer' | 'refactor'
  status     String @default("open") // open | merged | closed
  category   String // 'security' | 'tests' | 'refactor' | 'cleanup'

  diff        Json // [{ file, additions, deletions, changes }]
  aiExplanation String @db.Text

  checksStatus Json // { testsPassing: true, noConflicts: true, ... }

  additions    Int @default(0)
  deletions    Int @default(0)
  filesChanged Int @default(0)

  mergedAt    DateTime?
  closedAt    DateTime?

  bug         Bug?    @relation(fields: [bugId], references: [id])

  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@index([repositoryId])
  @@index([status])
}

```

Action Items:

1. Add PullRequest model to Prisma schema
 2. Create /api/pull-requests/* routes
 3. Convert HTML → React component
 4. Build diff viewer component with syntax highlighting
 5. Wire up merge/close buttons
 6. Add batch selection logic
-

6. Authentication (Not in your frontend yet)

Backend Already Has:

```
// Login
POST /api/auth/login
// Body: { email, password }

// Signup
POST /api/signup
// Body: { email, password, name, companyName }

// Session (NextAuth)
GET /api/auth/session
```

Action Items:

1. Create `/app/login/page.tsx` matching your design system
2. Create `/app/signup/page.tsx` matching your design system
3. Add auth checks to all dashboard pages
4. Add logout button to sidebar user section



Design System Migration

Tailwind Config Update

Your custom CSS needs to be converted to Tailwind config:

```
// tailwind.config.ts
export default {
  theme: {
    extend: {
      colors: {
        lime: '#c8ff00',
        'lime-dim': '#a3d000',
        ember: '#ff6b35',
        surface: {
          1: '#0a0a0a',
          2: '#0f0f0f',
          3: '#151515',
          4: '#1a1a1a',
          5: '#222222',
        },
      },
    },
  },
  fontFamily: {
    sans: ['Space Grotesk', 'sans-serif'],
    mono: ['JetBrains Mono', 'monospace'],
  },
  animation: {
    'pulse-dot': 'pulse-dot 2s ease-in-out infinite',
    'orbit': 'orbit 4s linear infinite',
    'scanning': 'scanning 1.5s linear infinite',
    'float': 'float 4s ease-in-out infinite',
  },
  keyframes: {
    'pulse-dot': {
      '0%, 100%': { opacity: 1, transform: 'scale(1)' },
      '50%': { opacity: 0.5, transform: 'scale(1.2)' },
    },
    orbit: {
      from: { transform: 'rotate(0deg) translateX(60px) rotate(0deg)' },
      to: { transform: 'rotate(360deg) translateX(60px) rotate(-360deg)' },
    },
    scanning: {
      '0%': { backgroundPosition: '0% 50%' },
      '100%': { backgroundPosition: '200% 50%' },
    },
    float: {
      '0%, 100%': { transform: 'translateY(0)' },
      '50%': { transform: 'translateY(-10px)' },
    },
  },
},
},
}
```

Component Library to Build

Create reusable components matching your design:

```

/components/
  /ui/
    - Badge.tsx (status badges)
    - ToggleSwitch.tsx (swarm on/off)
    - ProgressBar.tsx (with scanning animation)
    - StatCard.tsx (health scores, metrics)
    - SwarmCard.tsx (large cards on swarms page)
    - DiffViewer.tsx (code diff with syntax highlighting)
    - HealthRing.tsx (circular progress)
    - ActivityFeedItem.tsx (timeline items)
  /layout/
    - Sidebar.tsx (navigation)
    - TopBar.tsx (search, notifications, actions)
    - DashboardLayout.tsx (wrapper)
  /swarms/
    - SwarmStatusCard.tsx
    - SwarmConfigPanel.tsx
  /repositories/
    - RepositoryCard.tsx
    - LanguageBar.tsx
  /pull-requests/
    - PRLListItem.tsx
    - PRDetailView.tsx
    - DiffLine.tsx

```



Database Schema Additions Summary

New Models Needed:

1. ☒ **Repository** - Store connected repos
2. ☒ **SwarmConfig** - Per-org swarm settings
3. ☒ **PullRequest** - Track PRs created by agents
4. ☒ **ActivityFeed** - Live activity log (optional - can use AgentActivity)

Prisma Migration Command:

```

cd /home/ubuntu/shipfix/nextjs_space
yarn prisma migrate dev --name add_repositories_swarms_prs

```



Implementation Phases

Phase 1: Foundation (Week 1)

1. ☒ Update Tailwind config with your design tokens
2. ☒ Create base component library
3. ☒ Build DashboardLayout with sidebar & topbar
4. ☒ Set up authentication pages (login/signup)

Phase 2: Core Pages (Week 2-3)

1. ☒ Convert Dashboard page
 - Swarm status cards

- Activity feed
- Health metrics
- 2. ☒ Add Repository model & APIs
- 3. ☒ Convert Repositories page
 - List view
 - Connect new repo flow
- 4. ☒ Update database schema (Repository, SwarmConfig)

Phase 3: Advanced Features (Week 4)

- 1. ☒ Convert Swarms page
 - Swarm cards with toggles
 - Configuration panel
- 2. ☒ Add PullRequest model & APIs
- 3. ☒ Convert Pull Requests page
 - PR list
 - Diff viewer
 - Merge workflow

Phase 4: Real-time & Polish (Week 5)

- 1. ☒ Add Server-Sent Events for real-time updates
- 2. ☒ Wire up all agent triggers
- 3. ☒ Add error handling & loading states
- 4. ☒ Test all flows end-to-end
- 5. ☒ Deploy & validate



API Reference Quick Guide

Existing APIs (Use These)

```
// Authentication
POST /api/auth/login
POST /api/signup
GET /api/auth/session

// Bugs (Issues)
GET /api/bugs?organizationId={id}
GET /api/bugs/{id}
POST /api/bugs/upload // File upload for scanning

// Agents
POST /api/agents/triage // Scanner Agent
POST /api/agents/diagnostic // Fix generation
POST /api/agents/review // Pre-merge review
GET /api/agents/activity?bugId={id}

// Scanning
POST /api/scan/pre // Quick pre-scan (30s)
POST /api/scan/full // Full scan with constitution
```

New APIs to Build

```
// Repositories
GET /api/repositories
GET /api/repositories/{id}
POST /api/repositories/connect
POST /api/repositories/{id}/scan
GET /api/repositories/{id}/health

// Swarms
GET /api/swarms/status          // Real-time status of all swarms
GET /api/swarms/config
PATCH /api/swarms/config
POST /api/swarms/{type}/toggle
POST /api/swarms/{type}/start
POST /api/swarms/{type}/stop
GET /api/swarms/{type}/stats

// Pull Requests
GET /api/pull-requests
GET /api/pull-requests/{id}
POST /api/pull-requests/{id}/merge
POST /api/pull-requests/{id}/close
POST /api/pull-requests/batch-merge

// Activity Feed
GET /api/activity/feed?limit=10
```

Priority Mapping (What to Build First)

P0 - Critical Path (Build This Week)

1. ☒ Update Tailwind config
2. ☒ Build Sidebar + TopBar layout
3. ☒ Convert Dashboard page (basic version)
4. ☒ Add Repository model + basic CRUD APIs
5. ☒ Authentication pages

P1 - Core Features (Build Next Week)

1. ☒ Repositories page
2. ☒ Swarm configuration logic
3. ☒ Real-time updates (SSE)
4. ☒ Connect GitHub integration

P2 - Advanced (Build Week 3-4)

1. ☒ Pull Requests page
2. ☒ Diff viewer
3. ☒ Advanced swarm controls
4. ☒ Notification system



Key Technical Decisions

1. Real-time Updates

Use Server-Sent Events (SSE):

```
// app/api/swarms/status/stream/route.ts
export async function GET() {
  const encoder = new TextEncoder();

  const stream = new ReadableStream({
    async start(controller) {
      const interval = setInterval(async () => {
        const status = await getSwarmStatus();
        controller.enqueue(encoder.encode(`data: ${JSON.stringify(status)}\n\n`));
      }, 2000);

      return () => clearInterval(interval);
    },
  });

  return new Response(stream, {
    headers: {
      'Content-Type': 'text/event-stream',
      'Cache-Control': 'no-cache',
      'Connection': 'keep-alive',
    },
  });
}
```

Frontend Hook:

```
// hooks/useSwarmStatus.ts
export function useSwarmStatus() {
  const [status, setStatus] = useState(null);

  useEffect(() => {
    const eventSource = new EventSource('/api/swarms/status/stream');

    eventSource.onmessage = (event) => {
      setStatus(JSON.parse(event.data));
    };

    return () => eventSource.close();
  }, []);

  return status;
}
```

2. GitHub Integration

For “Connect Repository” feature:

```
// Use GitHub OAuth (already scaffolded)
// Or use Personal Access Token for MVP

POST /api/repositories/connect
Body: {
  gitUrl: 'https://github.com/acme/webapp-core',
  provider: 'github',
  accessToken: 'ghp_xxxx' // Optional for private repos
}
```

3. Diff Viewer

Use `react-diff-view` or build custom:

```
import { parseDiff, Diff } from 'react-diff-view';

const diffText = `
--- a/api/users.ts
+++ b/api/users.ts
@@ -46,2 +46,12 @@
-   const query = `SELECT * FROM users WHERE id = '${userId}'`;
+   // Sanitize input
+   const sanitizedId = sanitizeInput(userId);
`;

const files = parseDiff(diffText);

<Diff viewType="unified" diffType="modify" hunks={files[0].hunks} />
```

Design Optimizations & Recommendations

What's Already Great:

1.  **Color System** - Lime green (#c8ff00) is distinctive and eye-catching
2.  **Typography** - Space Grotesk + JetBrains Mono is a solid combo
3.  **Animations** - Orbital animation on landing page is 
4.  **Dark Theme** - Clean, modern, easy on the eyes
5.  **Component Hierarchy** - Clear visual separation with panels

Suggested Improvements:

1. Consistency Tweaks

Issue: Some status badges use different color schemes

Fix: Standardize badge colors:

```
const badgeColors = {
  security: 'bg-[#ff6b35]/20 text-[#ff6b35]', // Critical
  warning: 'bg-amber-500/20 text-amber-400', // High
  info: 'bg-white/10 text-white/60', // Low
  success: 'bg-emerald-500/20 text-emerald-400',
  active: 'bg-[#c8ff00]/20 text-[#c8ff00]',
};
```

2. Loading States

Add skeleton loaders:

```
// components/ui/SkeletonCard.tsx
export function SkeletonCard() {
  return (
    <div className="panel-raised rounded-2xl p-6 animate-pulse">
      <div className="h-6 bg-white/5 rounded w-1/3 mb-4"></div>
      <div className="h-4 bg-white/5 rounded w-2/3"></div>
    </div>
  );
}
```

3. Error States

Add empty states & error messages:

```
// components/EmptyState.tsx
export function EmptyState({ icon, title, description, action }) {
  return (
    <div className="flex flex-col items-center justify-center py-16 text-center">
      <div className="w-16 h-16 rounded-2xl bg-white/5 flex items-center justify-center mb-4">
        <i className={` ${icon} text-2xl text-white/30`}></i>
      </div>
      <h3 className="text-lg font-semibold mb-2">{title}</h3>
      <p className="text-white/40 mb-6 max-w-sm">{description}</p>
      {action}
    </div>
  );
}
```

4. Responsive Design

Current design is desktop-first. Add mobile breakpoints:

```
// Example: Swarm cards grid
<div className="grid grid-cols-1 md:grid-cols-2 gap-6">
  { /* Swarm cards */ }
</div>

// Sidebar: Hide on mobile, show hamburger menu
<aside className="sidebar hidden lg:flex">...</aside>
<button className="lg:hidden" onClick={toggleMobileMenu}>
  <i className="fas fa-bars"></i>
</button>
```

5. Accessibility

Add ARIA labels and keyboard navigation:

```
// Toggle switch
<button
  role="switch"
  aria-checked={isActive}
  onClick={() => setIsActive(!isActive)}
  className="toggle-switch"
>
  <span className="sr-only">Toggle swarm</span>
</button>
```

Testing Strategy

1. Component Testing (Vitest + React Testing Library)

```
// __tests__/components/SwarmCard.test.tsx
import { render, screen } from '@testing-library/react';
import { SwarmCard } from '@components/swarms/SwarmCard';

test('renders swarm card with correct stats', () => {
  render(
    <SwarmCard
      name="Scanner"
      status="running"
      progress={67}
      stats={{ issuesFound: 23, filesScanned: 847 }}
    />
  );

  expect(screen.getByText('Scanner')).toBeInTheDocument();
  expect(screen.getByText('67%')).toBeInTheDocument();
});
```

2. API Testing (Existing Pattern)

```
// __tests__/api/repositories.test.ts
import { POST } from '@app/api/repositories/connect/route';

test('connects repository successfully', async () => {
  const req = new Request('http://localhost:3000/api/repositories/connect', {
    method: 'POST',
    body: JSON.stringify({
      gitUrl: 'https://github.com/test/repo',
      provider: 'github',
    }),
  });

  const res = await POST(req);
  const data = await res.json();

  expect(res.status).toBe(200);
  expect(data.repository).toBeDefined();
});
```

3. E2E Testing (Playwright)

```
// e2e/dashboard.spec.ts
import { test, expect } from '@playwright/test';

test('user can navigate to repositories', async ({ page }) => {
  await page.goto('/dashboard');
  await page.click('text=Repositories');

  await expect(page).toHaveURL('/dashboard/repositories');
  await expect(page.locator('h1')).toContainText('Repositories');
});
```



Next Steps (Action Plan)

This Week (Dec 6-12)

Day 1-2: Foundation

- [] Update `tailwind.config.ts` with your colors & animations
- [] Create base layout components (Sidebar, TopBar)
- [] Set up component library structure

Day 3-4: Database & APIs

- [] Add `Repository`, `SwarmConfig`, `PullRequest` models to Prisma
- [] Run migrations
- [] Create `/api/repositories/*` endpoints
- [] Create `/api/swarms/*` endpoints

Day 5-7: Core Pages

- [] Convert Dashboard page to React
- [] Convert Repositories page to React
- [] Wire up basic data flow
- [] Test authentication flow

Next Week (Dec 13-19)

- [] Complete Swarms page
- [] Build Pull Requests page
- [] Add real-time updates (SSE)
- [] Connect GitHub OAuth
- [] End-to-end testing



Success Metrics

How to know if integration is successful:

1. ☒ User can log in and see dashboard
2. ☒ User can connect a repository
3. ☒ User can trigger a scan on a repository

4. ☒ Swarms run and show real-time progress
5. ☒ PRs are created and visible in PR page
6. ☒ User can merge/close PRs
7. ☒ All pages match your custom design
8. ☒ No TypeScript errors
9. ☒ All tests passing



Gotchas & Common Pitfalls

1. Static HTML → React

Problem: Direct copy-paste won't work due to:

- `class` → `className`
- Inline event handlers → React handlers
- Global styles → Scoped components

Solution: Use a converter or manually refactor:

```
# HTML to JSX converter
npx html-to-jsx < old.html > new.tsx
```

2. Font Awesome

Problem: CDN fonts won't work server-side.

Solution: Use `react-icons` or `@fortawesome/react-fontawesome`:

```
cd /home/ubuntu/shipfix/nextjs_space
yarn add react-icons
```

```
import { FaRadar, FaVial, FaWrench } from 'react-icons/fa6';

<FaRadar className="text-lime" />
```

3. Animations with Tailwind

Problem: Custom CSS animations need keyframes.

Solution: Add to `tailwind.config.ts` (see Design System section)

4. Real-time Updates Performance

Problem: Too many SSE connections can overload server.

Solution: Use single shared connection with context:

```
// contexts/SwarmStatusContext.tsx
const SwarmStatusContext = createContext(null);

export function SwarmStatusProvider({ children }) {
  const status = useSwarmStatus(); // Only one connection
  return (
    <SwarmStatusContext.Provider value={status}>
      {children}
    </SwarmStatusContext.Provider>
  );
}
```

Resources

Documentation:

- [Next.js App Router](https://nextjs.org/docs/app) (<https://nextjs.org/docs/app>)
- [Prisma Docs](https://www.prisma.io/docs) (<https://www.prisma.io/docs>)
- [Tailwind CSS](https://tailwindcss.com/docs) (<https://tailwindcss.com/docs>)
- [React Icons](https://react-icons.github.io/react-icons/) (<https://react-icons.github.io/react-icons/>)

Code Examples:

- [Server-Sent Events in Next.js](https://nextjs.org/docs/app/building-your-application/routing/route-handlers#streaming) (<https://nextjs.org/docs/app/building-your-application/routing/route-handlers#streaming>)
- [Prisma Migrations](https://www.prisma.io/docs/concepts/components/prisma-migrate) (<https://www.prisma.io/docs/concepts/components/prisma-migrate>)

Checklist (Before Launch)

Code Quality

- ☐ No TypeScript errors (`yarn tsc --noEmit`)
- ☐ All tests passing (`yarn test`)
- ☐ No console errors in browser
- ☐ Linting clean (`yarn lint`)

Functionality

- ☐ Authentication works (login/signup/logout)
- ☐ Repository connection works
- ☐ Scans trigger and show progress
- ☐ PRs are created and mergeable
- ☐ Real-time updates work
- ☐ All navigation links work

Design

- ☐ All pages match mockups
- ☐ Responsive on mobile/tablet/desktop
- ☐ Animations smooth

- [] Loading states implemented
- [] Error states implemented
- [] Empty states implemented

Performance

- [] Lighthouse score > 90
- [] First Contentful Paint < 1.5s
- [] Time to Interactive < 3s
- [] No memory leaks (SSE connections close properly)

You're Ready to Ship!

This roadmap covers **everything** you need to wire your beautiful frontend to the powerful ShipFix backend.

Questions? Let's build this! 